

Last week, in one line

Everything we do is estimating the f in $Y = f(X) + \epsilon$ — and what matters is **test error, not training error**.

Today's data: what is X, what is Y

Dataset	X (input · predictors)	Y (output · response)	Y is a...
Advertising	TV / radio / newspaper spend	sales	number → regression
Auto	horsepower	mpg	number → regression
Default	balance, income, student	default (Yes/No)	category → classification
Boston (lab)	lstat, age, ...	medv (median value)	number → regression
Smarket (lab)	last 5 days' returns, volume	Direction (Up/Down)	category → classification

- **X is what we already know** (and can sometimes control); **Y is what we want to know**
- The only fork in today's road: **Y is a number → Chapter 3; Y is a category → Chapter 4**. The machinery barely changes.

Chapter 3 · Regression

Predict a Number

Chapter 3 — the map

One dataset, `Advertising`, carries the whole chapter:

- **The model** — fit a line by least squares; read β_0, β_1 as statements about the business
- **Two spaces** — one line in data space = one point in parameter space; **fitting = finding the lowest point**
- **Scoring** — R^2 measures fit, but only context makes it *good*
- **More predictors** — `sales ~ TV + radio`: each coefficient holds the others fixed
- **Beyond lines** — polynomials on `Auto`: more flexible fits always look better on training data
- **Lab** — you refit every number yourself in `week2-lab`, Part A

The simplest useful model

Assume the relationship between X and Y is (approximately) a straight line:

$$Y \approx \beta_0 + \beta_1 X$$

For the `Advertising` data: $\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}$.

- A **parametric** method in its purest form: estimating f reduces to estimating just two numbers
- Dull? Maybe. But **it is often surprisingly hard to beat** — and by the end of this hour you will watch it beat a method that assumes nothing at all

Meet the data: Advertising

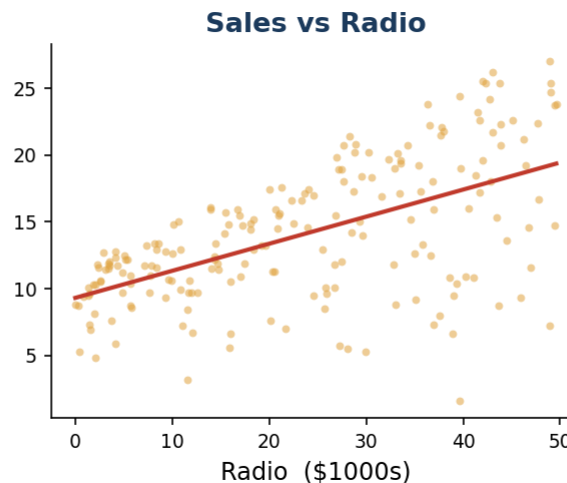
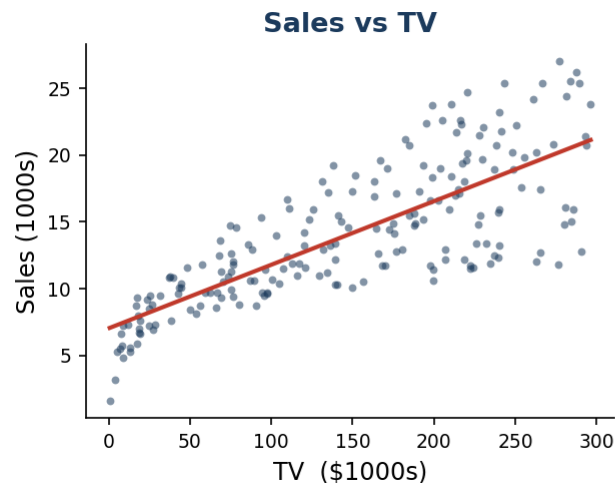
200 markets. Does spending on TV, radio, or newspaper ads move product sales? The running example for all of Chapter 3.

Inputs (X): TV · Radio · Newspaper ad budgets, in \$1000s

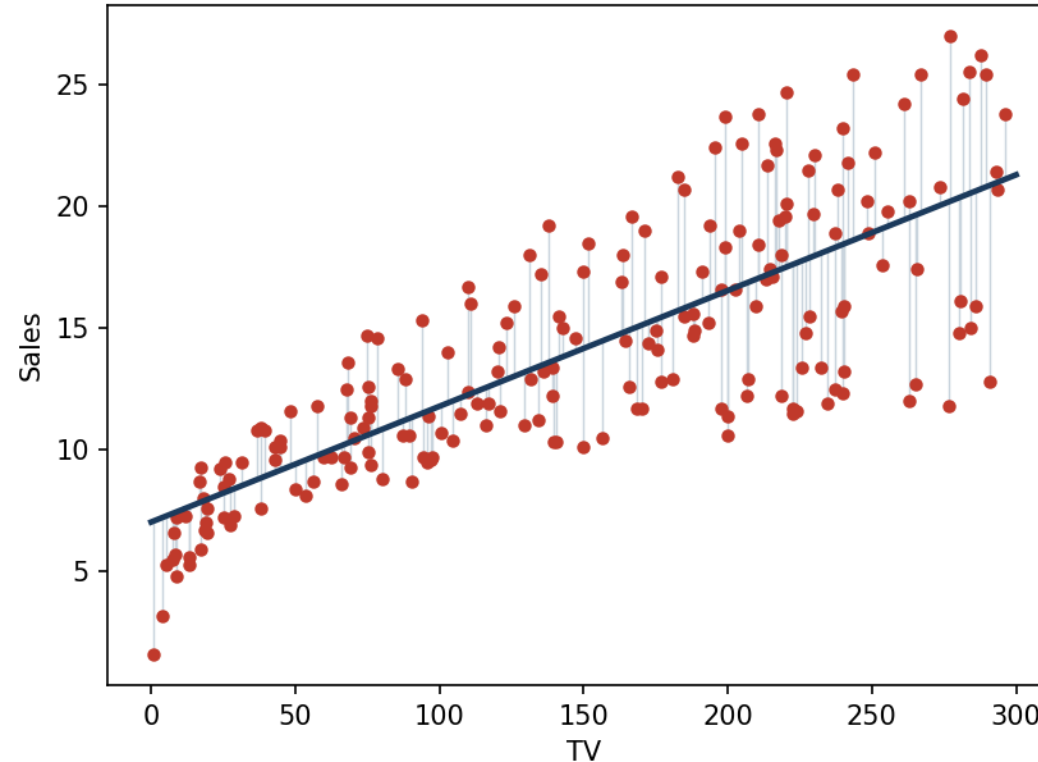
Output (Y): Sales, in 1000s of units → a **number**
→ **regression**

n = 200 markets · p = 3 predictors

TV	Radio	Newspaper	Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5

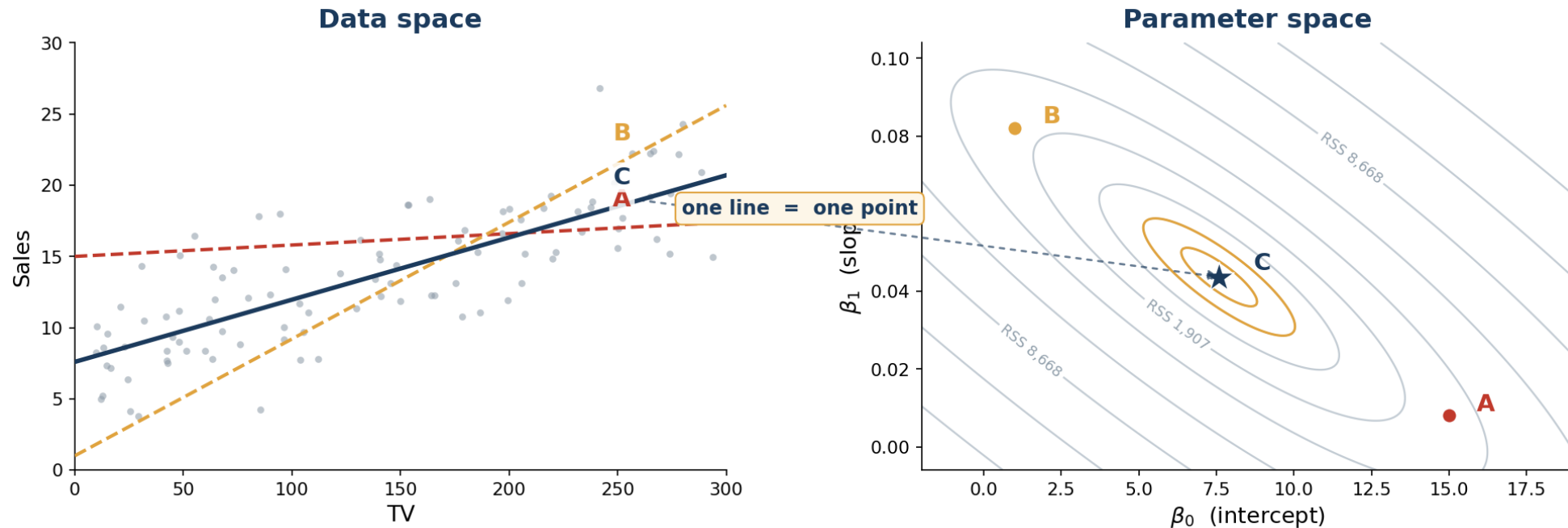


Least squares: how the line is chosen



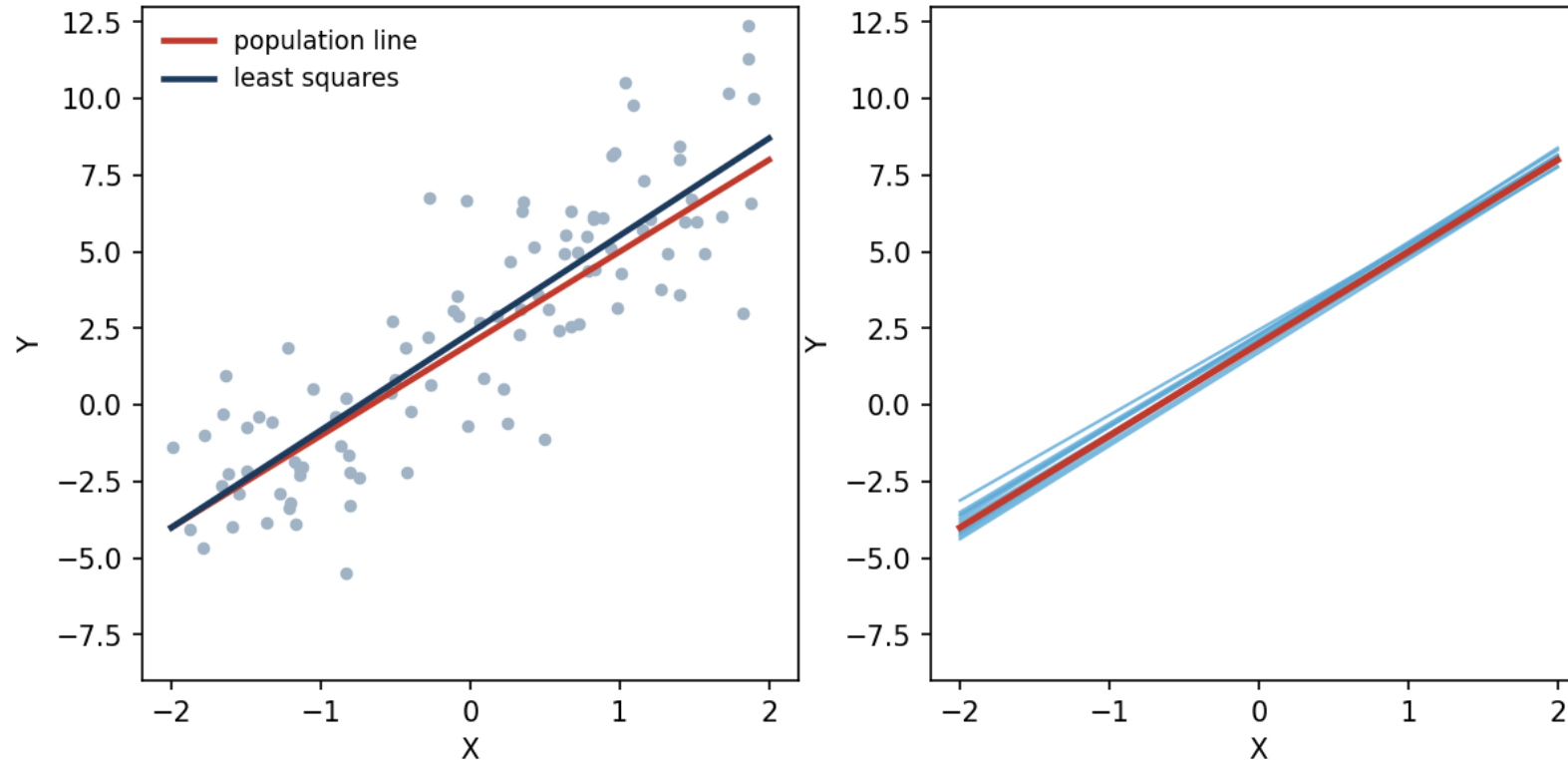
Each grey segment is a **residual** $e_i = y_i - \hat{y}_i$. Least squares picks the intercept and slope that minimize the **residual sum of squares** $RSS = e_1^2 + e_2^2 + \dots + e_n^2$.

Two spaces: one line = one point



Left (data space) — axes are X and Y : a dot is one observation, a line is one candidate model. **Right (parameter space)** — axes are β_0 and β_1 : **a point is an entire line from the left panel**; the contours are RSS. Lines A · B · C are points A · B · C — least squares C sits at the bottom of the bowl. Fitting = finding the lowest point on the right; **Week 5's neural networks descend a surface like this by gradient descent**. Read the axes first: $X, Y \rightarrow$ data space; β 's $\rightarrow$ parameter space.

Our line is an estimate — from one sample

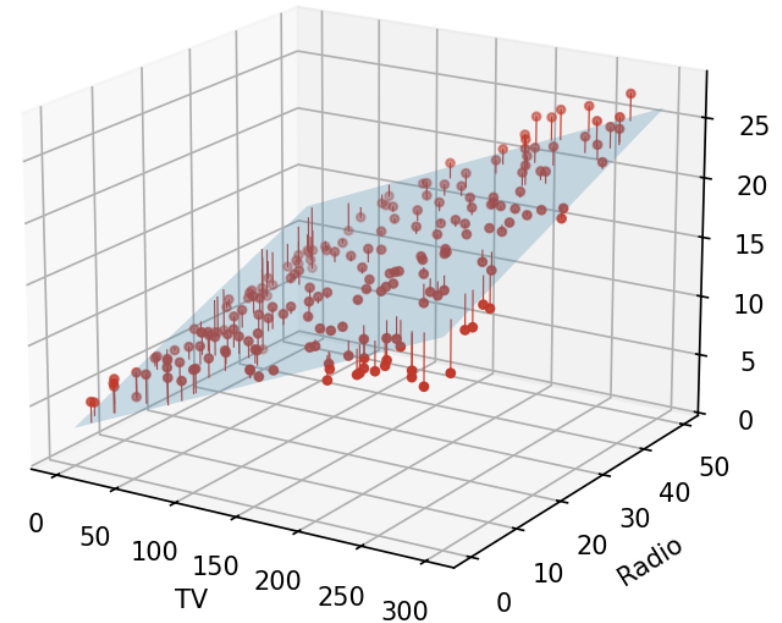


Red: the true *population* line. Dark blue: the least squares line from one sample. Light blue: least squares lines from ten other samples. Each line differs, but on average they land on the truth — **each sample gives a slightly different line**.

More than one predictor

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

- Each β_j : the average effect on Y of a one-unit increase in X_j , **holding all other predictors fixed**
- With two predictors, the fitted “line” becomes a plane; beyond that, a hyperplane

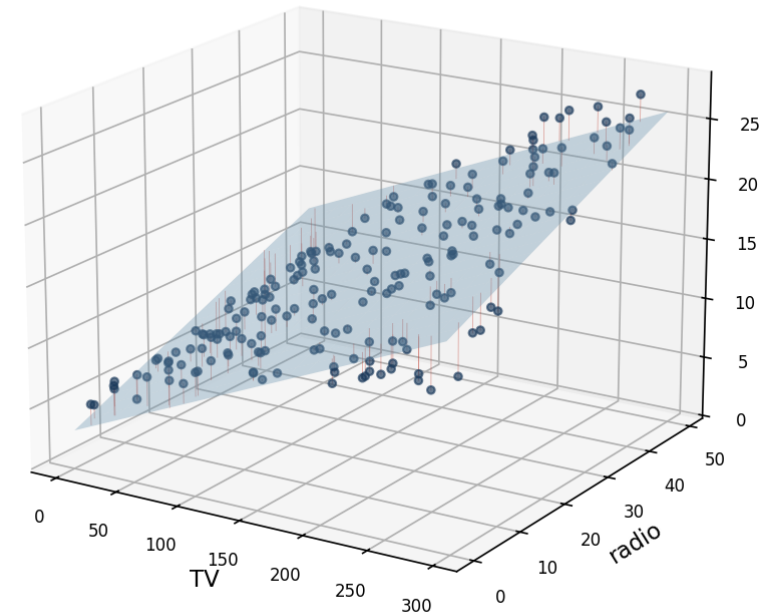


Two predictors in action: $\text{sales} \sim \text{TV} + \text{radio}$

Fit the plane by least squares on the real Advertising data:

$$\widehat{\text{sales}} = 2.92 + 0.046 \text{TV} + 0.188 \text{radio}$$

- The plane is the 2-predictor version of the line — each red stem is a **residual**, and least squares tilts the plane to minimise their squares
- Read a coefficient the right way: **+0.188** is the effect of one more \$1k of radio *at a fixed TV budget*
- Two predictors together explain $R^2 = 0.90$, well above TV alone ($R^2 = 0.61$)



$R^2 = 0.61$. **Is that good?**

The TV model explains **61%** of the variability in sales.

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

Good · Bad · Cannot tell

Hands up

Cannot tell — and that is the honest answer

R^2 has no universal scale. **Physics expects 0.99. Social science celebrates 0.3.** The same 0.61 is a triumph in one field and a red flag in another.

The honest move is to translate the miss into units the decision-maker cares about: look back at the gray residual segments — on a typical market the line misses by **a few thousand units of sales**. Now go ask a marketing manager whether that is acceptable.

A goodness-of-fit number is a question, not an answer. This will happen again in twenty minutes, with a much more dangerous number.

Meet the data: Auto

392 cars. How does an engine's power relate to its fuel economy?

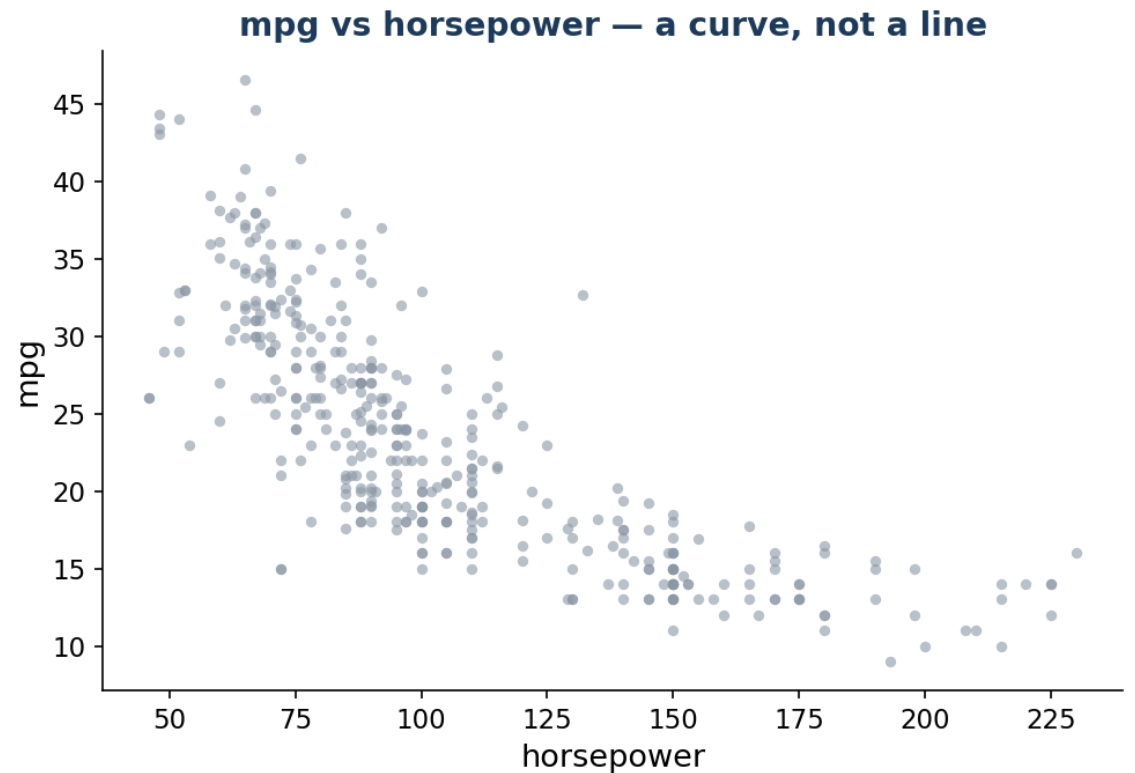
Inputs (X): all 7 remaining columns — cylinders, displacement, horsepower, weight, acceleration, year, origin

Output (Y): mpg (miles per gallon) → a **number** → **regression**

$n = 392 \text{ cars} \cdot p = 7 \text{ predictors}$

mpg	hp	weight	cyl
18.0	130	3504	8
15.0	165	3693	8
18.0	150	3436	8
16.0	150	3433	8

(mini-table previews 4 of the 8 columns)



Same model, higher powers

To bend the line, just add powers of `horsepower` as new columns of X :

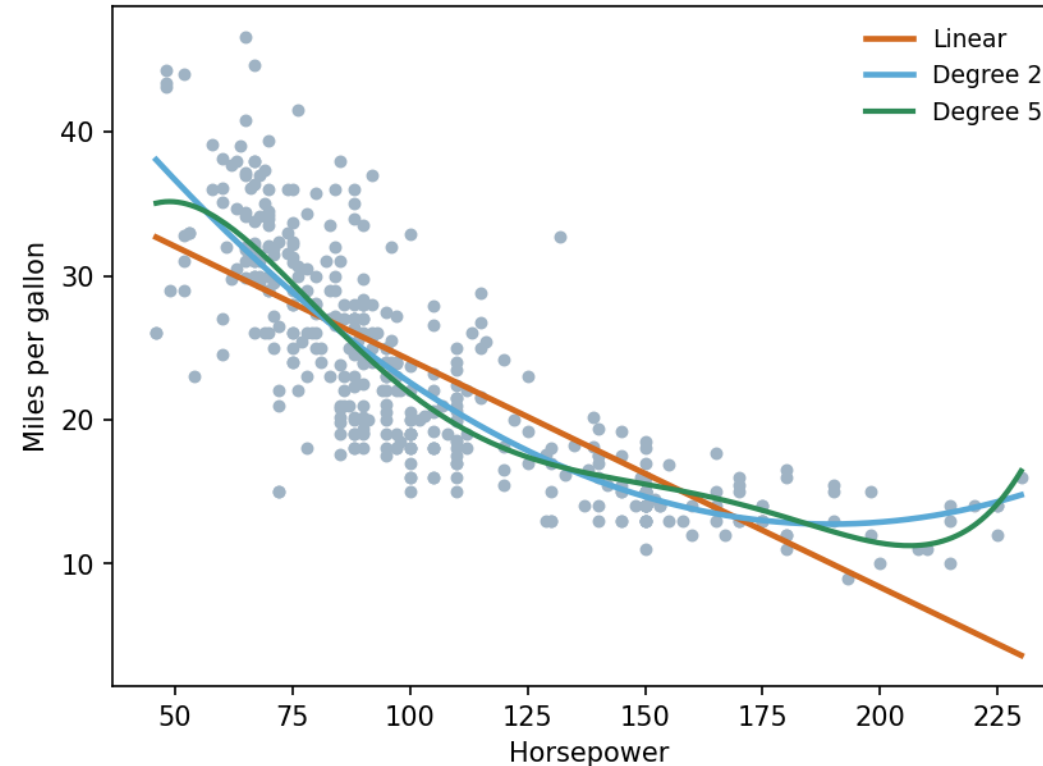
$$\text{degree 1: } \text{mpg} = \beta_0 + \beta_1 \text{hp}$$

$$\text{degree 2: } \text{mpg} = \beta_0 + \beta_1 \text{hp} + \beta_2 \text{hp}^2$$

$$\text{degree 5: } \text{mpg} = \beta_0 + \beta_1 \text{hp} + \beta_2 \text{hp}^2 + \dots + \beta_5 \text{hp}^5$$

- Every one of these is **still linear regression** — linear in the coefficients β . We are not fitting a curved model, we are fitting a straight one to extra columns
- Higher degree = more flexibility — the fit hugs the training data more tightly. The next slide shows what each degree actually does to the fit
- `I(horsepower**2)` in a formula is exactly this — one more column, nothing more exotic

Straight lines are not sacred



Degree 2 improves on linear. **Degree 5 starts chasing noise** — look at the right edge. Adding horsepower^2 is *still linear regression*, by the way: linear in the **coefficients**, not in X .

This week's lab, part 1: week2-lab , Part A

One file, zero setup: open `week2-lab.ipynb` in Jupyter or Colab and **Run All**. The first cell installs anything missing, and the `Advertising` data is embedded in the notebook itself — **nothing to download, nothing to configure**.

```
ADVERTISING = {"TV": [230.1, 44.5, ...], "Radio": [...], "Sales": [...]}
adv = pd.DataFrame(ADVERTISING)           # dict of lists -> a table
m1 = smf.ols("Sales ~ TV", data=adv).fit() # today's line, refit by you
```

- **A0–A1** — build the DataFrame from raw Python data, refit today's line
- **A3–A4** — recompute $R^2 = 0.61 \rightarrow 0.90$ with your own residuals
- Every headline number from these slides is reproduced in the notebook
- Going deeper (interactions, qualitative predictors): official ISLP lab 3.6

Regression, in three lines

1. Fit a line or plane by least squares (minimise RSS). A coefficient β_j is the effect of X_j **holding the other predictors fixed**.
2. R^2 says how much variation the model explains — but **only context says whether that is good**. Translate misses into units people care about.
3. Straight lines are not sacred: powers of X are still linear regression, and they always fit the training data better. **Whether they *predict* better is next week's question.**

Chapter 4 · Classification

Predict a Category

Chapter 4 — the map

One dataset again — `Default`, 10,000 credit-card customers:

- **The problem** — the output is a category (Yes/No), so we model a **probability**
- **The S-curve** — logistic regression keeps probabilities inside $(0, 1)$
- **Fitting** — the same two-space picture as Chapter 3, with a new surface
- **Reading results** — the confusion matrix, and why **97% accuracy can lie**
- **The threshold** — 0.3 vs 0.5 vs 0.7 flag very different customers; ROC / AUC compare them all
- **Lab** — `week2-lab`, Parts B–C; your threshold decision is homework #3

Meet the data: Default

10,000 credit-card customers. Will this person miss a payment? The running example for all of Chapter 4.

Inputs (X): balance, income, student (Yes/No)

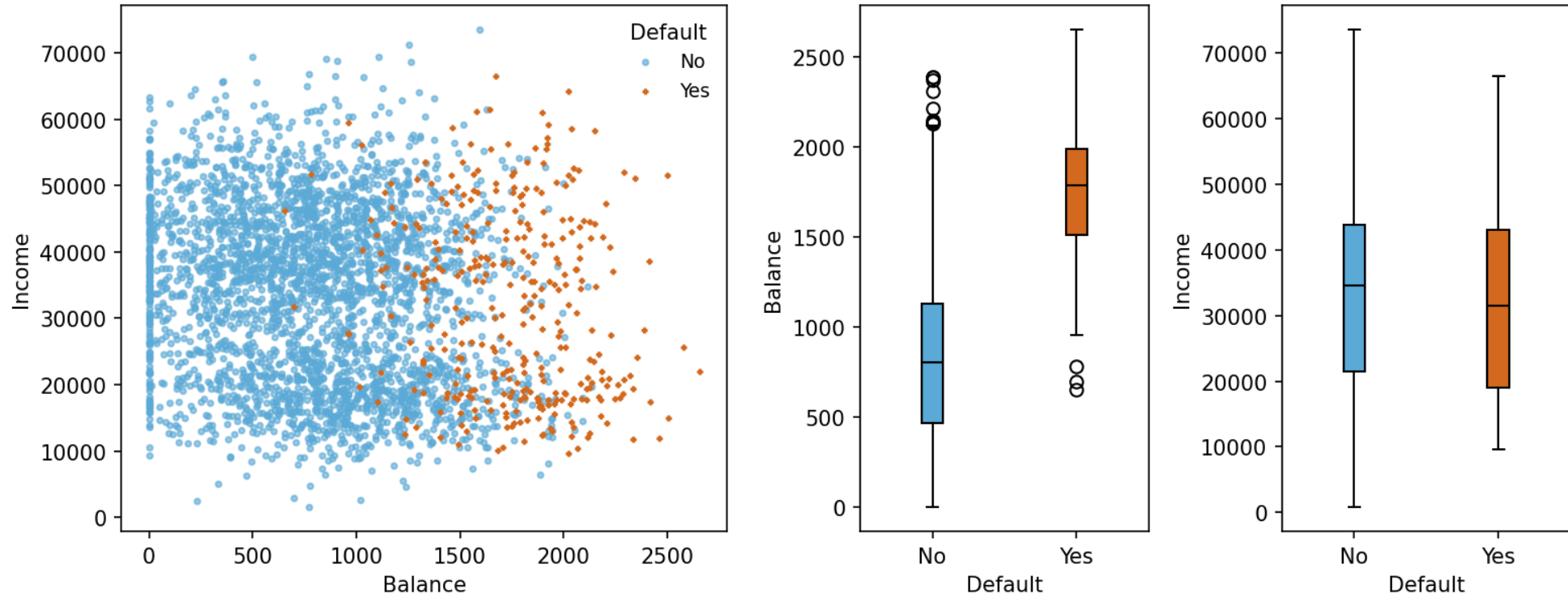
Output (Y): default — Yes/No → a **category** → **classification**

$n = 10,000$ · $p = 3$ · only 3.33% default

default	student	balance	income
No	No	730	44,362
No	Yes	817	12,106
Yes	Yes	1,487	17,854
Yes	Yes	2,206	14,271

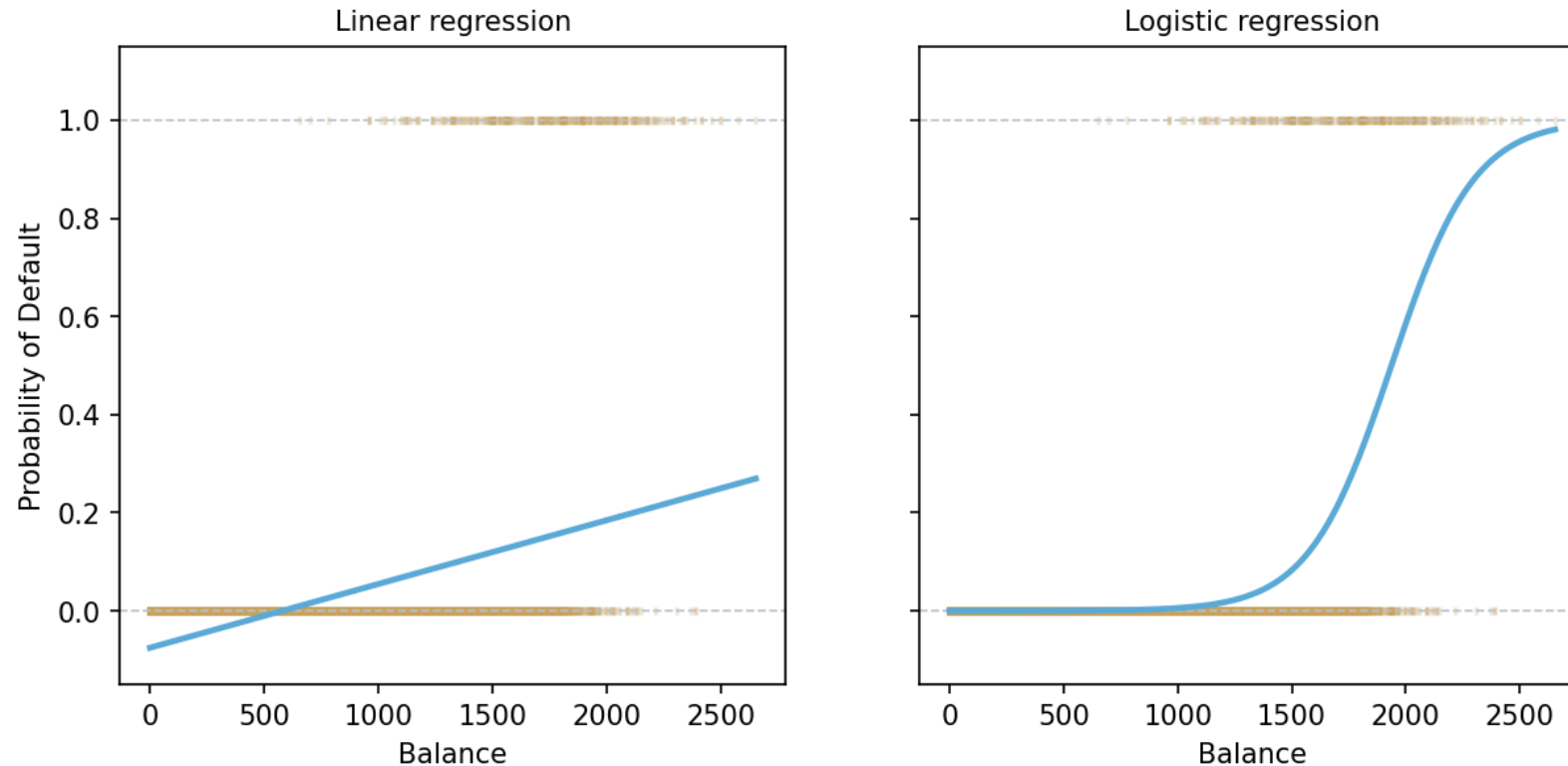


New problem: predicting a category



The `Default` data: will a customer **default** on their credit card debt? Balance clearly separates the orange (default) from blue (no default); income barely matters. The response is now **Yes/No — not a number**.

Why not just run linear regression?



Code No = 0, Yes = 1 and regress? The line (left) produces probabilities **below zero** and, for other data, above one. Logistic regression (right) squeezes the output into $(0, 1)$ — a proper probability.

The logistic model

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

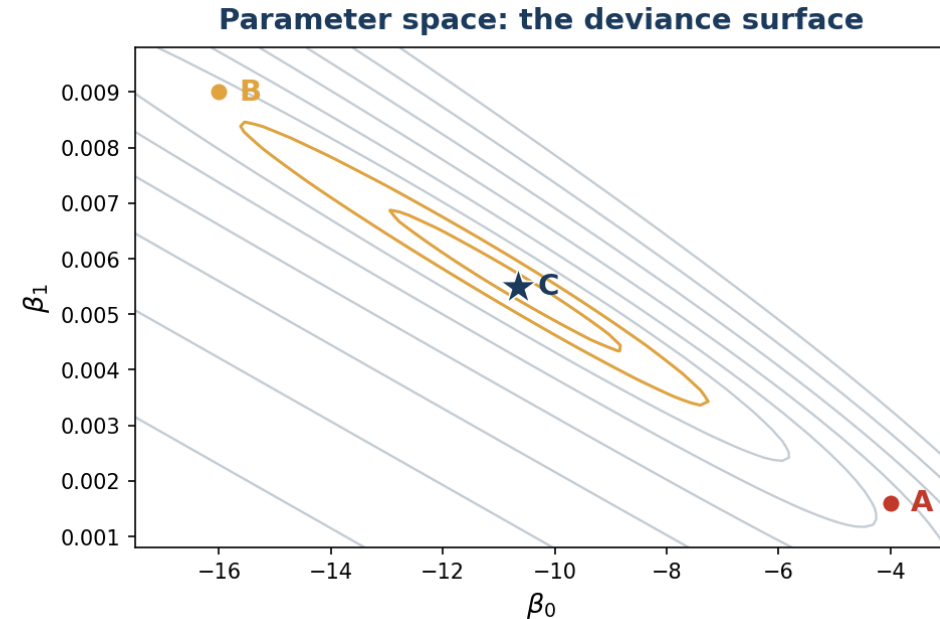
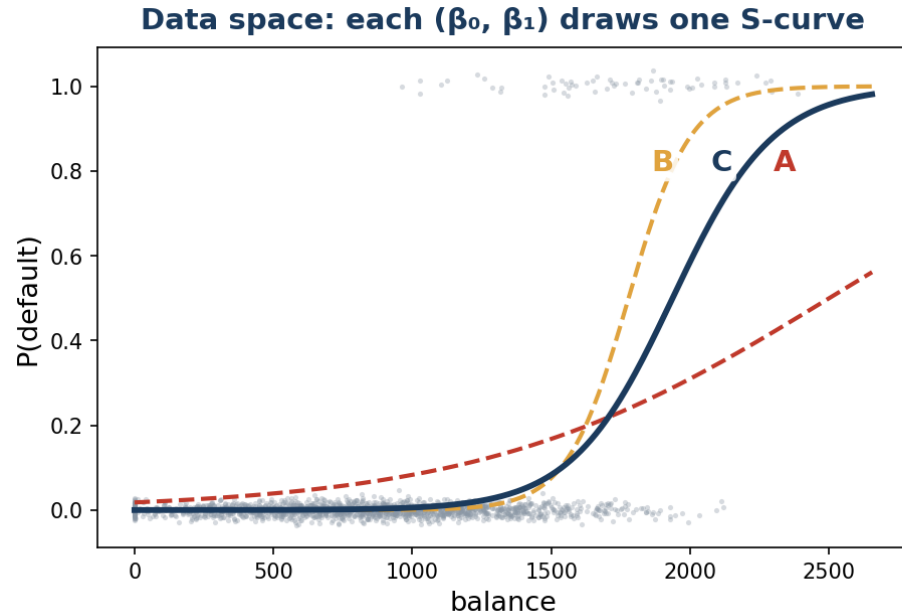
Rearranged, the **log odds** are linear in X :

$$\log \left(\frac{p(X)}{1 - p(X)} \right) = \beta_0 + \beta_1 X$$

One more unit of X adds β_1 to the log odds — the probability itself moves by a *varying* amount. That is the S-curve.

For $K > 2$ classes, the **softmax** generalisation treats every class symmetrically — and it is the exact function sitting at the output layer of every classification network you will meet in **Week 5**.

Fitting: the two-space picture again



Left: each (β_0, β_1) pair draws one S-curve through the data — **move β_1 and the curve steepens; move β_0 and it slides**. **Right:** the same pairs are points on the **deviance** surface (the misfit score that maximum likelihood minimises). Fitting = walking downhill to C: $\hat{\beta}_0 = -10.65$, $\hat{\beta}_1 = 0.0055$ — so balance \$1,000 $\rightarrow \hat{p} = 0.6\%$, \$2,000 $\rightarrow 58.6\%$.

The confusion matrix

Accuracy compresses everything into one number. The confusion matrix keeps all four:

	pred: no	pred: default
truth: no	TN — correctly cleared	FP — false alarm
truth: default	FN — the miss that costs money	TP — caught

- Rows are the truth; columns are what the model said. Every prediction lands in exactly one cell, so the four counts always sum to n
- The two error types are **not equal**: an FN is a real defaulter we cleared; an FP is a good customer we annoyed. **Which is worse is a business question, not a statistics question.**

Four numbers from the four cells

Every standard metric is a ratio of the same four counts:

metric	formula	the question it answers
Accuracy	$(TP + TN) / n$	how often are we right overall?
Precision	$TP / (TP + FP)$	of those we <i>flagged</i> , how many were real?
Recall (= TPR)	$TP / (TP + FN)$	of the <i>real</i> defaulters, how many did we catch?
F1	$2PR / (P + R)$	one number balancing precision and recall

- Accuracy **breaks on imbalanced data** — exactly our 3%-default problem
- Precision and recall **fight each other**: raising the threshold trades recall for precision — **the threshold fight you are about to watch**
- F1 is their harmonic mean: high only when *both* are high

Your turn: four numbers from these cells

A model screened **100 loan applications**:

	pred: no	pred: default
truth: no	50	10
truth: default	15	25

Accuracy? · Precision? · Recall? · F1?

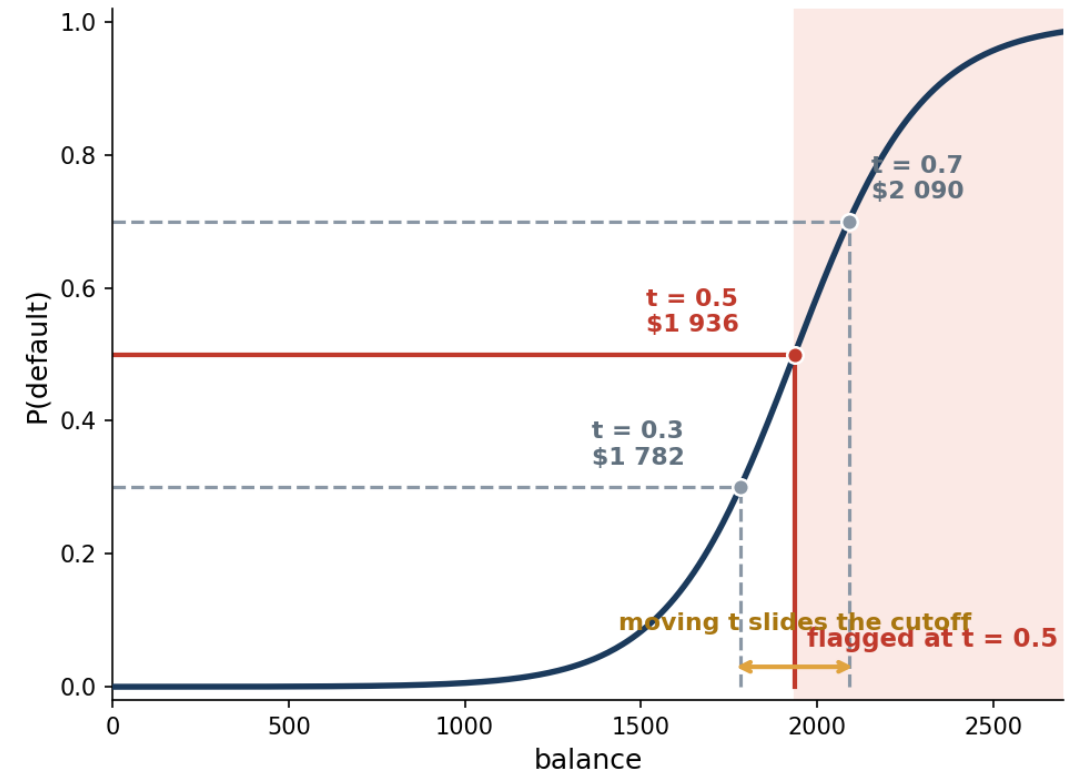
Work them out — hands up when you have all four

Our model at threshold 0.5

Logistic regression on `Default`; classify as *default* when $\hat{p} \geq 0.5$:

	pred: no	pred: default
truth: no	9,627	40
truth: default	228	105

- Accuracy **97.3%** — but **“nobody defaults”** scores **96.7%**
- Missed **228 of 333 defaulters** — **68.5%**
- Precision **72.4%** · Recall **31.5%** · F1 **0.44**



The threshold is a **horizontal line on the S-curve**: raise it and the balance cutoff slides right, flagging fewer people.

Move the threshold: 0.3 vs 0.7

Same model, same data — only the cut-off changes:

$t = 0.3$ — flag anyone above 30%

	pred: no	pred: default
truth: no	9,530	137
truth: default	164	169

missed defaulters **49.2%** · overall error **3.0%** ·
flagged **306**

$t = 0.7$ — flag only above 70%

	pred: no	pred: default
truth: no	9,655	12
truth: default	276	57

missed defaulters **82.9%** · overall error **2.9%** ·
flagged **69**

Lower the threshold and you catch defaulters but flood the review team; raise it and the error rate looks fine while defaulters walk through. **The model cannot pick for you — someone owns this choice.**

Two rates summarise any threshold

Every confusion matrix compresses to two rates. At threshold t :

	pred: no	pred: default
truth: no	TN	FP
truth: default	FN	TP

At $t = 0.5$: TN 9,627 · FP 40 · FN 228 · TP 105.

$$\text{TPR (recall)} = \frac{TP}{TP + FN} = 0.32$$

$$\text{FPR} = \frac{FP}{FP + TN} = 0.004$$

- **TPR** — of the real defaulters, the share we **catch** ($t = 0.3 \rightarrow 0.51$, $t = 0.7 \rightarrow 0.17$)
- **FPR** — of the good customers, the share we **falsely flag**
- Accuracy $= \frac{TP + TN}{n}$ looked great — and **we saw why that lies** on a 3%-default problem

Predict this one

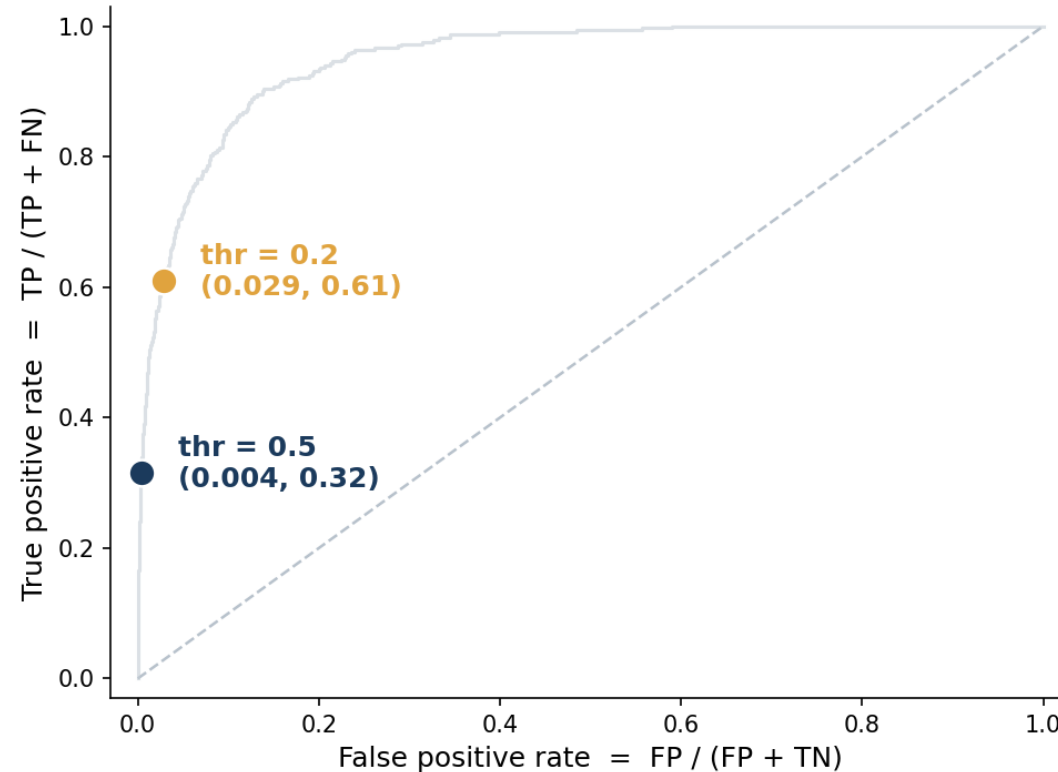
Right now the threshold is 0.5. Drop it to **0.2** — you flag more customers.

On the ROC plane (FPR $\rightarrow$, TPR $\uparrow$), the point moves...?

up and right · down and left · along the diagonal

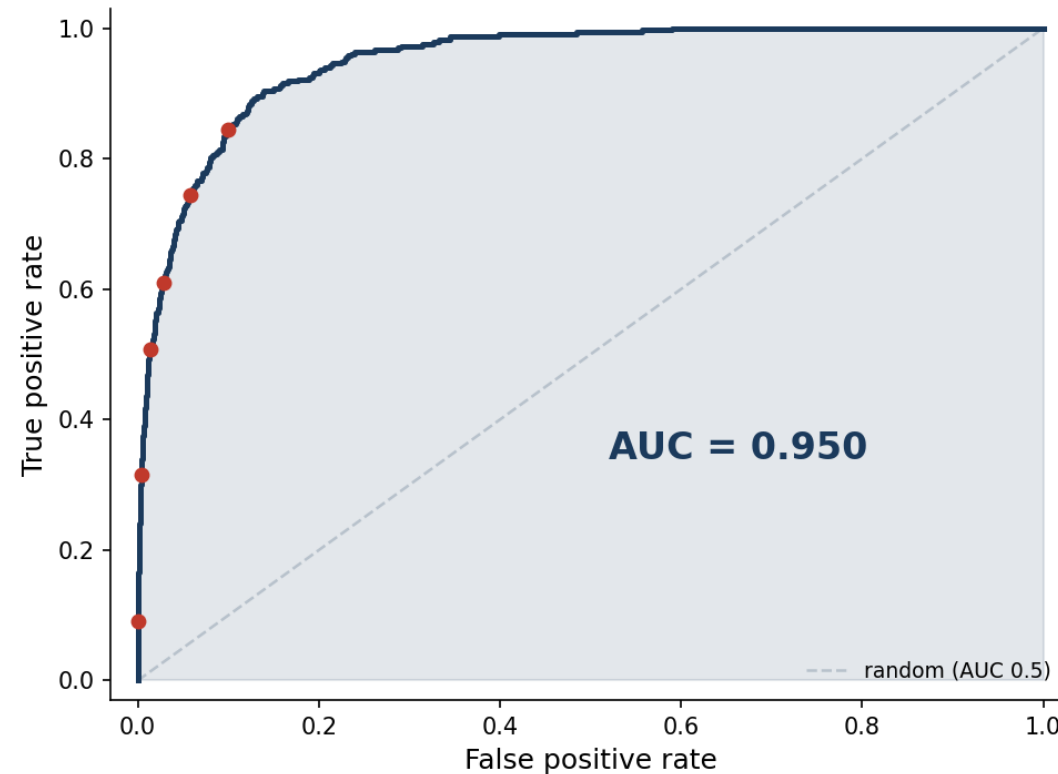
Commit before I click

One threshold = one point



Each threshold gives one confusion matrix → one (FPR, TPR) pair → **one point**. Lowering it to 0.2 catches more defaulters (TPR ↑) but raises false alarms (FPR ↑): **the point climbs up and to the right**.

The ROC curve: all thresholds at once



Do that for **every** threshold and the points trace the curve (red dots = the ones above). The **AUC** is the area under it — a single, threshold-free score, here **0.95**. Read it as: the chance the model scores a random defaulter above a random non-defaulter. The closer to the top-left corner, the better.

This week's lab, part 2: week2-lab , Parts B–C

Same notebook, same **Run All**. Part B rebuilds everything you just saw on `Default` — the 0.5 matrix, the threshold comparison, ROC / AUC — and **cell B5 is homework #3**:

```
THRESHOLD = 0.20      # <- change me, rerun, repeat  
# prints your confusion matrix, missed-defaulter %, and error rate
```

Part C points the same machinery at the stock market (`Smarket`) and lands **barely above a coin flip** — not a bug in your code; the most honest result in the lab. **A model that reports it knows nothing has told you something true.**

- Optional extras (LDA, QDA, naive Bayes, KNN): official ISLP lab 4.7

Classification, in three lines

1. Predict a category by modelling $\Pr(Y|X)$: **logistic regression** squeezes a linear model through the S-curve into $(0, 1)$ — a proper probability, fitted by walking downhill on the deviance surface.
2. One accuracy number can lie. Read the **confusion matrix** — the two error types cost **different money**.
3. The **threshold** is a business decision: 0.3, 0.5, 0.7 flag very different customers. Compare them all at once with **ROC / AUC**.

Before Next Friday

Homework

1. Read **ISLP Ch. 5–6** ([week3/ch05-06-resampling-regularization.pdf](#)) — Resampling and Regularization, for Jul 24
2. Run this week's labs: **3.6** (Linear Regression — [Boston](#) , [Carseats](#)) and **4.7** (Logistic Regression — [Smarket](#) ; the notebook also covers LDA, QDA, naive Bayes, KNN as optional extras)
3. **The threshold question.** On [Default](#) , pick the threshold you would actually recommend to a credit card company. One paragraph defending it. **There is no right answer, and I will push back on whatever you choose.**
4. Next week: how to *validate* every model you fit today. Cross-validation is the tool you will use for the rest of your career.

Questions?

Every number that looks good is a question, not an answer.

$R^2 = 0.61$. Accuracy = 97.3%.

See you next Friday